

Ты хочешь **сначала построить “машину проектирования”** (как мы работаем с ИИ, где лежат файлы, как фиксируются решения, кто проверяет кого), и **только потом** — проектировать сам SPA. Это правильно: иначе вы утонете в чатах, противоречиях и “потерянных” версиях.

Ниже — фундаментальная архитектура: **какие модели, что они делают, как они взаимодействуют с документами, и готовые промты** (копируй-вставляй).

1) Базовая архитектура работы с ИИ

1.1. Single Source of Truth

Единственный источник правды = папка в Google Drive (Markdown-документы + логи).

Чаты/модели — это “процессоры”, но **решения живут в файлах**.

Почему так: любой ИИ “забывает контекст” и начинает дрейфовать. Файлы — якорь.

1.2. “Хаб” в ChatGPT

В ChatGPT создаёшь **Project** под SPA и используешь его как:

- место, где лежат ключевые файлы,
- единые кастомные инструкции,
- цепочка обсуждений по теме.

1.3. Подключение Google Drive к ChatGPT

Чтобы не грузить файлы руками каждый раз, подключаешь **Google Drive app (sync)** в ChatGPT:

- Профиль → Settings → **Apps** → Google Drive → Connect → Sync.

Важные ограничения:

- синк “по-взрослому” обычно требует **Google Workspace домен** (gmail.com / googlemail.com не поддерживаются для sync).
- В ChatGPT “connectors” сейчас называются **Apps** (переименование было в декабре 2025).

- Есть режим “подтягивания по запросу” (access) и режим “sync”. Для проектирования лучше sync, потому что у тебя постоянные изменения.
-

2) Роли моделей и правила взаимодействия

Ниже — “кто за что отвечает”, чтобы не было каши.

ChatGPT 5.2 Pro —

Архитектор-интегратор (Owner of Truth)

Что делает:

- пишет **первую версию** документов (политики, спецификации, шаблоны),
- интегрирует правки от Claude/Gemini,
- следит за непротиворечивостью всей системы,
- ведёт **changes.log** и Decision Records (ADR).

Что НЕ делает:

- не принимает решения “на ощущениях”, если не зафиксирована политика/ограничения в документе,
- не хранит секреты (seed, приватные ключи).

Claude —

Редтим + ясность текста

Что делает:

- ищет дырки: двусмысленности, противоречия, опасные формулировки,
- проверяет, что друг “с улицы” поймёт текст правильно,
- предлагает правки как “патч”.

Что НЕ делает:

- не пишет финальную версию политики (только ревью),
- не придумывает новые требования без привязки к контексту.

Gemini Pro —

Инженер тестов + крайние случаи

Что делает:

- генерирует тест-матрицы, edge cases, “как это сломается”,
- придумывает измеримые KPI и критерии готовности,
- предлагает альтернативные архитектуры (2–3 варианта) и сравнение.

Что НЕ делает:

- не редактирует финальный документ напрямую (только рекомендации).

OpenCLAW —

Оркестратор/автоматизация (не “мозг”)

Что делает:

- запускает регламентные задачи (сбор данных, мониторинг, обновление индексов),
- делает “док-операционку”: проверяет, что у документа есть шапка, версия, статус, ссылки,
- позже — исполняет on-chain операции **по правилам**, а не “по вдохновению”.

Что НЕ делает:

- не принимает риск-решения сам,
 - не имеет доступа к ключам без твоего железного процесса (hardware/roles).
-

3) Как мы фиксируем знания, чтобы не упереться в лимит памяти

3.1. Статусы документов

У каждого документа в шапке:

- **Status:** DRAFT / REVIEW / FROZEN / SUPERSEDED
- **Version:** v0.1, v0.2 ...
- **Owner:** кто главный за этот документ
- **Last updated:** дата
- **Depends on:** список документов, на которые опирается

3.2. Правило изменения

Любое изменение политики или архитектуры:

- создаёт запись в **changes.log**
 - (опционально) создаёт ADR “почему мы так решили”
-

4) Набор “фундаментальных” промтов для каждой ИИ

Ниже — готовые промты. Их можно хранить в Drive как /prompts/*.md, чтобы всегда были одинаковые.

4.1. ChatGPT 5.2 Pro — “SPA Architect / Integrator”

Системные инструкции (для Project instructions или Custom GPT):

Роль: Архитектор-интегратор проекта SPA. Ты пишешь и поддерживаешь набор документов проекта.

Источник правды: только файлы в папке SPA (Google Drive). Если данных нет в файлах — помечай как допущение или запрашивай недостающий документ.

Запрещено: хранить/просить seed phrases, приватные ключи, API-ключи.

Стиль вывода: всегда выдавай (1) краткое резюме, (2) основной текст документа/патча, (3) список открытых вопросов, (4) запись для changes.log.

Всегда соблюдай Risk Policy и Mode Policy как высший закон. Если их ещё нет — сначала создавай их.

Рабочий “стартовый” промт на сессию (первое сообщение в чате):

Проект SPA. Работаем по документ-центричному процессу.

Задача: создать/обновить документ <DOC_NAME> в папке /docs.

Формат документа: Markdown, с шапкой (Title, Version, Status, Owner, Last updated, Depends on), далее разделы, далее Change log.

Если не хватает входных документов — перечисли что нужно и какие допущения делаешь.

4.2. Claude — “SPA Red Team / Clarity”

Ты — редтим и редактор ясности документов SPA.

Вход: текст документа (Markdown) + его название/версия.

Выход:

- 1) Список критичных проблем (ambiguity, contradiction, missing definitions, unsafe promises).
 - 2) Список средних проблем (clarity, structure, reader confusion).
 - 3) Предлагаемый патч: “до/после” или bullet rewrite по разделам.
 - 4) 5–10 вопросов, которые нужно закрыть, чтобы документ стал FROZEN.
- Никаких новых фиц “с потолка”: только то, что улучшает корректность и читабельность.

4.3. Gemini Pro — “SPA Edge Cases / Test Engineer”

Ты — инженер тестов и крайних случаев для SPA.

Вход: (a) документ или схема, (b) цель/ограничения, (c) контекст режима (Stable / Stable+BTC+ETH).

Выход:

- 1) Таблица “Failure mode → Trigger → Expected behavior → Metrics → Test method”.
- 2) Список 20 edge cases (с приоритетом P0/P1/P2).
- 3) Минимальный набор метрик для мониторинга (не более 15, но самые нужные).
- 4) 2–3 альтернативные архитектуры/подхода, с плюсами/минусами.

4.4. OpenCLAW — “Job Specs” (как он будет использовать LLM)

Тут важнее не “промт”, а регламент вызова.

Шаблон задания для OpenCLAW (в YAML/JSON, как удобно):

```
job: doc_update
inputs:
  doc_path: "docs/03_Risk_Policy.md"
  change_request: "Добавить лимиты для режима Stable+BTC+ETH"
steps:
  - run: "lint_doc_header"
  - run: "call_llm"
    model: "gpt-5.2-pro"
    prompt_template: "prompts/gpt_architect_doc_patch.md"
```

```
- run: "call_llm"
  model: "claude"
  prompt_template: "prompts/claude_redteam.md"
- run: "call_llm"
  model: "gemini"
  prompt_template: "prompts/gemini_testcases.md"
- run: "merge_suggestions"
- run: "write_changes_log"
```

outputs:

```
- updated_doc
- review_notes
- changes_log_entry
```

Смысл: OpenCLAW не “думает”, он гоняет пайплайн: черновик → редтим → тесты → слияние → лог.

5) Что лучше: Project, Custom GPT, или оба?

Вариант А — проще всего и почти всегда хватает на старте

- **ChatGPT Project** = единый рабочий контур (чаты + файлы + инструкции).
- **Google Drive app with sync** для доступа к документам.
- Claude/Gemini — отдельные чаты, их выводы кладёте в /reviews/.

Плюс: минимум настройки.

Минус: “роли” держатся дисциплиной, а не интерфейсом.

Вариант В — структурно (если хочешь прям “по ролям” в ChatGPT)

- Создаёшь **несколько Custom GPT**: Architect, Redteam, Test Engineer, Scribe.
- Подключаешь знания/файлы (но помни про лимиты knowledge: до 20 файлов на GPT).
- Если нужно, делаешь Actions / интеграции (для технич. команды).
- Для Google Drive можно делать через Actions (dev-путь).

Нюанс: доступ “Apps в custom GPT” может зависеть от плана (часто это Business/Enterprise/Edu в beta).

6) Мой “усиленный контроль ошибок” в ответах и документах

Чтобы реально снизить ошибки (а не просто обещать), предлагаю встроить правило в процесс:

Каждый ответ/документ проходит 4 проверки:

1. **Consistency check:** не противоречит ли уже зафиксированным политикам/определениям.
2. **Math/logic check:** формулы/условия окупаемости/лимиты — проверка на здравый смысл.
3. **Scope check:** не добавил ли я фичу, которую вы не просили.
4. **Assumption labeling:** всё, что не доказано документами — помечается как допущение.

Плюс внешний контроль: Claude (ясность+дыры), Gemini (крайние случаи).

7) Практический “фундаментальный” план настройки ИИ-архитектуры (по шагам)

1. В Google Drive создаёшь папку SPA/ и подпапки:
 - docs/, prompts/, reviews/, logs/, adr/, archive/.
2. В ChatGPT подключаешь Google Drive app with sync (если доступно в твоём плане/домене).
3. Создаёшь ChatGPT **Project “SPA”** и добавляешь туда:
 - короткие инструкции (роль Architect/Integrator),
 - docs/00_Context.md и docs/INDEX.md (когда появятся).
4. Кладёшь в prompts/ 3 файла: gpt_architect.md, claude_redteam.md, gemini_tests.md (из раздела выше).

5. Дальше любые проектные обсуждения → только через выпуск/изменение документов, а не “в чате договорились”.

Если хочешь, следующий логичный шаг *в рамках этой “ИИ-архитектуры”* — я сформирую первый фундаментальный документ:

prompts/INDEX.md (что где лежит, какие роли, какие команды) +

docs/INDEX.md (карта всех документов, их статус, последняя версия).

Это будет ваш “центр управления полётами”, чтобы через 3 месяца не вспоминать, где правда.